Chika-Ekweghariri Munachiso Henry - 2312011009.

```
types.
    CustomerId = token;
    ManagerId = token;
    MenuItemId = token;

    Category = <Drinks> | <MainMeal> | <Swallow> | <Protein> | <Snacks>;

    MenuItem:: id : MenuItemId
               name: seq of char.
               category: Category
               price: nat
               quantityInStock: nat.
               description: seq of char;

    Carts = map MenuItemId to nat;

    Order:: Customer: CustomerId
            items : map MenuItemId to nat.
            total : nat;

state Cafeteria of
    registeredCustomers : set of CustomerId
    registeredManagers: set of ManagerId
    menu: map MenuItemId to MenuItem.
    carts: map CustomerId to Cart
    Orders: set of Order

inv mk_cafeteria(regCust, regMgr, m, c, ord) ==
    dom c subset regCust and.
    (forall cid in set dom c and dom c(cid) subset dom m) and
    (forall o in set ord and o.customer in set regCust and
```

```
dom o. items  subset dom m) '

init mk_Cafeteria (regCust, regMgr, m, c, ord) ==
     regCust = {} and regMgr = {} and m = {|->} and c = {|->}
     and ord = {}
end.

operations

  RegisterCustomer(cid: CustomerId).
  ext wr registeredCustomers: set of CustomerId.
      wr carts : map CustomerId to cart.
  pre cid not in set registeredCustomers~
  post registeredCustomers = registeredCustomers~ union {cid} and
  carts = carts~ ++ {cid |-> {|->}};

  ViewMenu() result: map MenuItemId to MenuItem
  ext rd menu: Map MenuItemId to MenuItem.
  post RESULT = menu;

  AddToCart(cid: CustomerId, mid: MenuItemId, qty: nat)
  ext wr carts : map CustomerId to cart
      rd menu: map MenuItemId to MenuItem
      rd registeredCustomers: set of CustomerId.
  pre cid in set registeredCustomers and
      mid in set dom menu and
      qty > 0 and
      (if mid in set dom carts~(cid)
      then carts~(cid)(mid) + qty <= menu(mid).quantityInStock
      else qty <= menu(mid).quantityInStock)
  post let oldCart = carts~(cid),
```

~~newQty = if mid in~~
newQty = if mid in set dom OldCart
then ~~Old~~ oldCart(mid) + qty
else qty
in
carts = carts~ ++ {cid |-> oldCart ++ {mid |-> newQty}};

PlaceOrder (cid: CustomerId) :
ext wr carts: map CustomerId to Cart.
wr menu: map MenuItemId to MenuItem
wr orders: set of Order.
rd registeredCustomers: set of CustomerId.
pre cid in set registeredCustomers and.
carts~(cid) <> {|->} and
(forall mid in set dom carts~(cid) and carts~(cid)(mid)
<= menu~(mid).quantityInStock).
post let OrderItems = carts~(cid),
totalAmount = sum mid in set dom orderItems and
menu~(mid).price * orderItems(mid)
in
orders = orders~ union {mk_Order(cid, orderItems, totalAmount)}
and menu = menu~ ++ {mid |-> mu(menu~(mid),
quantityInStock := menu~(mid).quantityInStock
- orderItems(mid)).
| mid in set dom OrderItems} and
carts = carts~ ++ {cid |-> {|->}};

AddMenuItem(mgrId: ManagerId, mid: MenuItemId, name:
seq of char, cat: Category; nat qtyInStock: nat, desc: seq of
chair)

```
ext wr menu: map MenuItemId to MenuItem
    rd registered Managers : set of ManagerId.
pre mgrId in set registeredManagers and
    mid not in set dom menu and
    price > 0 and
    qtyInStock >= 0
post menu = menu~ ++ {mid |-> mk_Menu_Item(mid,
name, cat, price, qtyInStock, desc)};

end
```